

A Quantum Algorithm for Zig-Zag Functions, with Applications to Wall–Sun–Sun Primes, and a Grover-Based Algorithm for the Golden Mean

M.Chowdhury

Written 4th June 2025*

Revised 3rd September 2026*

Draft version

(Dates of server time stamps)*

Abstract

We propose a quantum algorithm for computing the Pisano period $\pi(m)$ of the Fibonacci sequence modulo an integer m . The Fibonacci recurrence modulo m is not injective within a single period, so the standard quantum period-finding template cannot be applied to it directly. We resolve this by constructing an auxiliary function F''_n ; built from a finite family of shifted, generalized (a, b) -Fibonacci sequences interleaved into a single integer by place value in a base B -which we call the *Zig-Zag function*, and which we conjecture is injective on one period once the number of interleaved terms U grows polynomially in $\log m$. We call this the *Zig-Zag Conjecture*. Conditional on this conjecture, standard quantum phase estimation applied to F''_n recovers $\pi(m)$ in time and space polynomial in $\log m$; we call this conditional result the *Quantum Zig-Zag Theorem*. We give numerical evidence for the conjecture in a worked example. As an immediate corollary, testing a candidate prime p for the Wall–Sun–Sun property ($p^2 \mid \pi(p)$) or the near-Wall–Sun–Sun congruence becomes a single classical check appended to our algorithm. In other words we present a quantum algorithm that can test for a Wall-Sun-Sun prime in quantum polynomial time and quantum polynomial space. Separately, and independently of the Pisano-period result above, we present a new (though not asymptotically faster than classical) hybrid classical–quantum numerical integration technique, based on a non-standard use of Grover’s algorithm together with a two-dimensional Weyl equidistribution argument, and use it to evaluate closed-form integral–series expressions for the golden ratio φ , Catalan’s constant G , and the beta function $B(n, n)$. We present this integration technique as a conceptually new construction in its own right, not as a complexity result. We describe a quantum algorithm to compute Catalan’s constant.

1 Introduction

In this paper we assume the widely used conjecture that an arbitrary quantum circuit cannot be simulated in polynomial time on a classical only computer. We also assume in ideal noise free(unless otherwise stated) universal gate quantum computer. The Fibonacci sequence is

defined by $F_0 = 0$, $F_1 = 1$, $F_n = F_{n-1} + F_{n-2}$. For a positive integer m , the sequence $(F_n \bmod m)_{n \geq 0}$ is periodic; its period $\pi(m)$ is the *Pisano period* of m . Pisano periods have been studied since Lagrange observed in 1877 that the Fibonacci sequence modulo 10 repeats with period 60. In this paper we refer to Pisano periods as Zig Zag type periods. Zig Zag functions in this paper produce Zig Zag type periods (including Pisano periods). Within one Pisano period there are 1 or 2, or 4 zeros; the count is called the *order* of m , and the position of the first zero is the *rank* of m . We will use the standard, unconditional bound $\pi(m) \leq 6m$ for all $m \geq 1$ (with equality iff $m = 2 \cdot 5^k$ for some k), so $\pi(m) = O(m)$ throughout. A search speedup for computing the rank would require, e.g., using Grover amplification over candidates which uses our Zig-Zag function as an oracle which we do not describe further in this paper (the above quantum algorithm to compute the rank).

A prime p is a *Wall–Sun–Sun (WSS) prime* if $p^2 \mid \pi(p)$. No WSS primes are known, though infinitely many are conjectured to exist. Prior to Wiles’s proof of Fermat’s Last Theorem, it was known that exhibiting a WSS prime would produce a counter example to Fermat’s Last Theorem [2], which historically motivated searches for them. A *near-WSS prime* satisfies

$$F_{p-(p/5)} \equiv Ap \pmod{p^2}, \quad (1)$$

for some integer A , where the Legendre-type symbol is

$$(p/5) = \begin{cases} 1 & p \equiv 1, 4 \pmod{5} \\ -1 & p \equiv 2, 3 \pmod{5}. \end{cases} \quad (2)$$

Contribution and scope. This paper has two independent parts, and we are careful to keep their claims separate:

1. **A conditional polynomial-time quantum algorithm for $\pi(m)$** (Sections 3–4). The obstruction to a direct application of quantum period finding is that $n \mapsto F_n \bmod m$ is not injective on one period, so we build an auxiliary injective-on-conjecture function F_n'' out of several shifted (a, b) -Fibonacci sequences. The resulting polynomial-time claim (Theorem 11) is *explicitly conditional* on Conjecture 5 below, for which we give numerical evidence but no proof. Wall–Sun–Sun and near-WSS testing (Section 5) is a direct, essentially "free corollary" of this algorithm, applied to one candidate at a time.
2. **A new hybrid classical–quantum numerical integration method** (Section 2), used to evaluate closed-form expressions for φ , Catalan’s constant, and the beta function. We present this as a conceptually new construction, not a speedup: we do not claim it is faster than the (other known classical, polynomial-time) classical algorithms that already exist for these constants. Its interest is that it is an intrinsically quantum way of extracting numerical values from Grover’s algorithm, rather than using Grover’s algorithm to search.

We use the (a, b) -generalized Fibonacci sequence throughout, defined by $F_0 = a$, $F_1 = b$, $F_n = F_{n-1} + F_{n-2}$, which reduces to the ordinary Fibonacci sequence when $(a, b) = (0, 1)$.

Writing $\varphi = \frac{1+\sqrt{5}}{2}$ and $\psi = \frac{1-\sqrt{5}}{2}$ for the two roots of $r^2 - r - 1 = 0$, the closed form is

$$F_{n,a,b} = \frac{b - a\psi}{\sqrt{5}} \varphi^n + \left(a - \frac{b - a\psi}{\sqrt{5}}\right) \psi^n. \quad (3)$$

Since $\psi = \frac{1}{2} - (\varphi - \frac{1}{2})$, evaluating (3) only requires φ to the desired precision.

2 A Hybrid Classical–Quantum Construction for the Golden Mean

2.1 An integral–series formula for φ

We use the identity

$$\varphi = \frac{11\pi}{200 \int_0^{\pi/2} \frac{dx}{(5 - 4 \cos^2 x)^3}} + \frac{1}{2} + \frac{11}{20} \sum_{n=0}^{\infty} \frac{\binom{2n}{n}}{5^{3n}}, \quad (4)$$

The proof of above involves a series term that is a hypergeometric-type sum; the integral term is what we use to illustrate the quantum construction below. The integral term is the one we work through in detail below to illustrate the quantum construction; the series term is not independent of it, however, in the following sense: the central binomial coefficients $\binom{2n}{n}$ appearing in it can themselves be obtained from the beta function via the classical identity

$$B(\zeta, \zeta) = \frac{\Gamma(\zeta)^2}{\Gamma(2\zeta)} = \frac{2/\zeta}{\binom{2\zeta}{\zeta}} \iff \binom{2\zeta}{\zeta} = \frac{2/\zeta}{B(\zeta, \zeta)}, \quad (5)$$

for positive integers ζ , and $B(n, n)$ itself has the same Grover-amenable integral form used throughout this section (see (6) below): so the same quantum construction that evaluates the integral term can, via $B(n, n)$ and (5), also supply the binomial coefficients in the series term, rather than these being computed by an unrelated classical means. We do not carry this substitution through numerically here, since it does not change the non-speedup status of the construction (Remark 2 applies equally to this use); we record the connection because it is what motivates treating φ , $B(n, n)$, and (by the same technique, see below) m Catalan’s constant as one coherent construction rather than three unrelated worked examples. In other words there is entirely analogous integral representation for the beta function $B(n, n)$ at integer arguments, via

$$B(n, n) = \int_0^{\pi/2} \frac{(\sin \theta \cos \theta)^{2n}}{\sin \theta \cos \theta} d\theta, \quad (6)$$

and we compute all by the same technique a Grover based numerical integration; we work through φ ’s integral in detail and observe note that the Catalan’s by the same construction with different integrands; computing Catalan’s constant is not the main subject of this paper we mention it here to express our Grover based numerical integration is a new quantum method to compute Catalans constant (or more generally constants that have representation that can be computed with our Grover based numerical integration.)

2.2 Motivation: why Grover’s algorithm at all

This construction is presented as an independent result and, as Remark 9 makes precise, is not used by the period-finding algorithm of Section 4, which instead computes all Fibonacci-type quantities by exact integer arithmetic. Our aim here is not to beat the classical composite trapezium rule (CTRD) for evaluating $\int_0^{\pi/2} F(\cos x, \sin x) dx$; that rule is standard and already polynomial-time classically. Our aim is to show that the values $\cos(x_i)$ at the quadrature nodes x_i can instead be recovered from Grover’s algorithm, by treating a Grover rotation angle as an *irrational rotation* and using it, together with a second, independent Grover rotation angle, to hit an arbitrary target node x_i via simultaneous Diophantine approximation. This is a genuinely different route to the same numbers, of independent interest as a quantum construction,

An entirely analogous integral representation exists for Catalan’s constant G and for the beta function $B(n, n)$ at integer arguments, via

$$B(n, n) = \int_0^{\pi/2} \frac{(\sin \theta \cos \theta)^{2n}}{\sin \theta \cos \theta} d\theta, \quad (7)$$

and we compute all three by the same technique; we work through φ ’s integral in detail and simply note that the other two follow by the same construction with different integrands.

2.3 On the Grover Based Numerical Integration Subroutine

This construction is presented as an independent result and, as Remark 9 makes precise, is not used by the period-finding algorithm of Section 4, which instead computes all Fibonacci-type quantities by exact integer arithmetic. Our aim here is not to beat the classical composite trapezium rule (CTRD) for evaluating $\int_0^{\pi/2} F(\cos x, \sin x) dx$; that rule is standard and already polynomial-time classically. Our aim is to show that the values $\cos(x_i)$ at the quadrature nodes x_i can instead be recovered from Grover’s algorithm, by treating a Grover rotation angle as an *irrational rotation* and using it, together with a second, independent Grover rotation angle, to hit an arbitrary target node x_i via simultaneous Diophantine approximation. This is a genuinely different route to the same numbers, of independent interest as a quantum construction, which is why we present it even though it carries no speed advantage; in fact, as discussed below, the number of Grover iterations required grows *exponentially* in the number of bits of precision required, so this sub-algorithm is, on its own, exponential time. It is included purely for its conceptual novelty, not for any complexity claim. This mirrors a broader point in the quantum algorithms literature: for example in the paper [7], it is expressed a quantum construction can be worth presenting for what it reveals about representing and manipulating a problem on a quantum computer even absent a proven speedup over the best classical method,

2.4 Construction

Run two independent instances of Grover’s algorithm, G_1 on a search space of 2^{Q_1} elements and G_2 on a search space of 2^{Q_2} elements ($Q_1 \neq Q_2$), each with a single marked solution (say, $x = 23$, which can be checked in polynomial time by any convenient oracle). The initial

angle between the “good” and “bad” subspaces in instance i is $\theta_i = 2 \arcsin(2^{-Q_i/2})$; because $\arcsin$ of a rational is generically irrational (indeed usually transcendental), θ_1 and θ_2 are, for generic Q_1, Q_2 , irrational and rationally independent: $\theta_1 \neq c\theta_2$ for any $c \in \mathbb{Q}$.

Step 1 (quantum): table generation via Quantum Phase Estimation. For a large integer bound κ , run G_1 for $k_1 = 1, \dots, \kappa$ iterations and G_2 for $k_2 = 1, \dots, \kappa$ iterations. For each iteration count, apply Quantum Phase Estimation (QPE) to the corresponding Grover operator (using the standard sine-shift circuit of Appendix A to recover the sine as well as the cosine branch) to measure the resulting angles

$$\Phi_{1,k_1} = (2k_1 + 1)\theta_1 \bmod 2\pi, \quad \Phi_{2,k_2} = (2k_2 + 1)\theta_2 \bmod 2\pi \quad (8)$$

directly from the quantum state, together with $\cos \Phi_{i,k_i}$ and $\sin \Phi_{i,k_i}$. This builds a quantum-generated lookup table, for each instance, of κ rows indexed by k_i ; crucially, the angle Φ_{i,k_i} is obtained by physically applying the composite rotation k_i times and reading it out with QPE, never by classically multiplying k_i by a stored finite-precision value of θ_i .

Step 2 (classical): Exhaustive search for a target node. To hit a specific target quadrature node x_i , pass both length- κ tables to a classical computer, which performs an $O(\kappa^2)$ exhaustive search over all pairs (k_1, k_2) , computing for each pair

$$\cos(\Phi_{1,k_1} + \Phi_{2,k_2}) = \cos \Phi_{1,k_1} \cos \Phi_{2,k_2} - \sin \Phi_{1,k_1} \sin \Phi_{2,k_2}, \quad (9)$$

via the table entries (full derivation of the sine-shift and angle-addition identities used here is given in Appendix A), and records the residual $|(\Phi_{1,k_1} + \Phi_{2,k_2}) \bmod 2\pi - x_i|$ until the accuracy within the residual is met-so it is like a brute force algorithm because it has the above stopping criteria. This means that the stopping criteria may exit the exhaustive search subroutine for a target node early if the stopping criteria is met early.

Step 3: node approximation. Because θ_1, θ_2 are irrational and rationally independent, the two-dimensional Weyl equidistribution theorem guarantees that the set

$$\{(\Phi_{1,k_1} + \Phi_{2,k_2}) \bmod 2\pi : 1 \leq k_1, k_2 \leq \kappa\} \quad (100)$$

becomes dense in $[0, 2\pi)$ as $\kappa \rightarrow \infty$; hence for every target x_i and every $\delta > 0$ there is a κ large enough that the Step 2 search finds a pair (k_1, k_2) with residual under δ . The matched pair's, within the residual/error bound, quantum-evaluated value $\cos(\Phi_{1,k_1} + \Phi_{2,k_2})$ is then slotted directly into the classical CTRD quadrature rule as $\cos(x_i)$.

Remark 1 (Absence of classical scaling errors). *A key conceptual feature of this construction is that the classical computer never needs to store θ_1, θ_2 to high precision. Were a classical machine with finite floating-point precision to compute the large multiples $(2k_i + 1)\theta_i \bmod 2\pi$ directly, the truncation error in the stored value of θ_i would scale linearly with k_i , and for the large k_i that high-precision Diophantine approximation requires, this would destroy the accuracy of the phase modulo 2π entirely. In our construction, the quantum computer instead physically applies the composite rotation via repeated Grover iterations, an operation with no*

arithmetic truncation on an ideal, noiseless machine, and QPE measures the resulting phase directly; the classical side only ever stores the finitely many already measured table entries, never the underlying irrational θ_i to arbitrary precision. This is a statement about representability, not about asymptotic complexity: as Remark 2 makes precise, the construction is exponential time regardless.

Remark 2 (This construction is not polynomial time - and worse than simple exponential). By Dirichlet's simultaneous approximation theorem, achieving residual $\delta \sim 2^{-n}$ in two dimensions generically requires κ as large as $\sim 2^{2n}$. Each of the κ table entries in Step 1 is itself obtained from a QPE call that must resolve n bits of the angle Φ_{i,k_i} , costing $O(2^n)$ controlled Grover-operator applications on its own (standard QPE precision cost). These two exponential factors compound rather than coexist: building both length- κ tables costs $\kappa \cdot O(2^n) = O(2^{2n} \cdot 2^n) = O(2^{3n})$ Grover oracle calls, and Step 2's classical search over all κ^2 pairs costs a further $O(\kappa^2) = O(2^{4n})$ comparisons. So the true cost is $O(2^{4n})$, worse than a bare $O(2^n)$ exponential in the number of bits n of desired precision. We do not claim, and it is not true, that this construction gives a faster way to evaluate (4) than the classical CTRD. It is included as a new intrinsically-quantum construction, for the reason given in Remark 1.

Once the node values $\cos(x_1), \dots, \cos(x_n)$ are tabulated this way, the CTRD quadrature rule

$$\int_0^{\pi/2} F(\cos x, \sin x) dx = \sum_{i=1}^n \frac{h_i}{2} (F(x_{i-1}) + F(x_i)) + \text{Error}_{\text{CTRD}}, \quad \text{Error}_{\text{CTRD}} = \frac{1}{12} \sum_i h_i^3 F''(\zeta_i), \quad (10)$$

evaluates $\int_0^{\pi/2} \frac{dx}{(5-4\cos^2 x)^3}$ and hence, via (4), φ . The construction for $B(n, n)$ and for Catalan's constant is the same, specialized to a single free parameter and equally spaced nodes; we omit the parallel derivation. Full step-by-step algebra for all cases is collected in Appendix A.

3 The Zig-Zag Function and Conjecture

3.1 Why the naive approach fails

Standard quantum period finding (as in Shor's algorithm) requires a function that is injective, at least, on a single period; applying the quantum Fourier transform to $n \mapsto F_n \bmod m$ directly fails because this map is generally not injective within one Pisano period.

3.2 Why the components can be stacked: a period-compatibility lemma

The stacking construction below only makes sense if the periods of the individual (a, b) -Fibonacci sequences being interleaved are compatible with $\pi(m)$ rather than arbitrary. This compatibility is not a conjecture since it follows from the following lemma about the Fibonacci matrix.

Lemma 3 (Period compatibility). *Let $A = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$. For every integer $m \geq 1$ and every pair of integers (a, b) , the sequence $(F_{n,a,b} \bmod m)_{n \geq 0}$ is purely periodic with period dividing $\pi(m)$.*

Proof. Writing $v_n = \begin{pmatrix} F_{n+1,a,b} \\ F_{n,a,b} \end{pmatrix}$, the recurrence gives $v_n = A^n v_0 \pmod{m}$ with $v_0 = \begin{pmatrix} b \\ a \end{pmatrix}$. Since $\det A = -1$ is a unit mod m , A is invertible mod m , hence has finite order in $\text{GL}_2(\mathbb{Z}/m\mathbb{Z})$; let t denote this order. Applying this to the ordinary Fibonacci sequence itself ($a = 0, b = 1$, i.e. $v_0 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$) and noting $Av_0 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$, we have $A^t v_0 = v_0$ and hence also $A^t \begin{pmatrix} 1 \\ 1 \end{pmatrix} = A^t Av_0 = AA^t v_0 = Av_0 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$, so A^t fixes both $\begin{pmatrix} 1 \\ 0 \end{pmatrix}$ and $\begin{pmatrix} 1 \\ 1 \end{pmatrix}$; since $\det \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} = 1$ is a unit, these two vectors form a basis of $(\mathbb{Z}/m\mathbb{Z})^2$, so A^t fixing both forces $A^t = I$. Thus $t \mid \pi(m)$, and by the defining minimality of $\pi(m)$ (the smallest period of the standard Fibonacci sequence mod m) in fact $t = \pi(m)$: the order of $A \bmod m$ equals $\pi(m)$ exactly. Finally, for arbitrary (a, b) , $A^{\pi(m)} v_0 = v_0$ for every vector v_0 (since $A^{\pi(m)} = I$), so $(F_{n,a,b} \bmod m)$ is periodic with period dividing $\pi(m)$. $\square$

This is precisely the fact that allows the construction: every component (a, b) -sequence we interleave into F_n'' automatically has a period dividing $\pi(m)$, so no interleaving of finitely many such components, or of shifts $F_{n+J, \cdot}$ of them (a shift does not change a sequence's period), can ever produce a combined period *larger* than $\pi(m)$; the only risk is a combined period *smaller* than $\pi(m)$, if every interleaved component happens to share some proper common divisor of $\pi(m)$ as its period. Guarding against exactly that risk is the role of the conjecture below.

3.3 Construction

Fix an integer base $B \geq m$ and constants $c_{K,n}$ (which may be taken to be 0 for simplicity). For $J = 0, 1, \dots, U$ define the block

$$Z_{J,n} = B^{3J} \left(\sum_{K=1}^2 \left[((B^K)F_{n+J,K+J+1,K+J+1}) + ((B^3)F_{n+J+1,3(J+1),3(J+1)}) \right] \right), \quad (11)$$

(with the constants $c_{K,n}$ added mod- B into each bracketed term when nonzero), and define the *Zig-Zag function*

$$F_n'' = Z_{0,n} + Z_{1,n} + \dots + Z_{U,n}. \quad (12)$$

This interleaves $U+1$ shifted (a, b) -Fibonacci terms into disjoint digit blocks of a single base- B integer, so that F_n'' records the values of several Fibonacci-type sequences simultaneously; we call each block $Z_{J,n}$ a “tooth,” and the resulting construction the paper’s namesake Zig-Zag construction, for a reason made precise below (see “Where the name comes from,” after Example 4). By Lemma 3, F_n'' is guaranteed to be periodic with period exactly dividing $\pi(m)$; the open question is whether it achieves $\pi(m)$ exactly rather than a proper divisor, which, as noted after Lemma 3, is equivalent to F_n'' being injective on one period of length $\pi(m)$: a period equal to a proper divisor $d \mid \pi(m)$, $d < \pi(m)$, would force F_n'' to repeat every d steps and hence fail to be injective on a window of length $\pi(m)$, and conversely.

Example 4. For $m = 3$, $U = 2$, $B = 10$, $c_{K,n} \equiv 0$, direct computation (verified numerically for $n = 1, \dots, 20$) gives the six component sequences

$$\begin{aligned} F_{n,1,1}: & \quad 01120221 \ 01120221 \ 01120221 \ \dots \\ F_{n,2,2}: & \quad 02210112 \ 02210112 \ 02210112 \ \dots \\ F_{n+1,3,3}: & \quad 00000000 \ 00000000 \ 00000000 \ \dots \\ F_{n+1,4,4}: & \quad 01120221 \ 01120221 \ 01120221 \ \dots \\ F_{n+1,5,5}: & \quad 02210112 \ 02210112 \ 02210112 \ \dots \\ F_{n+2,6,6}: & \quad 00000000 \ 00000000 \ 00000000 \ \dots \end{aligned}$$

each individually of period dividing $\pi(3) = 8$ (in fact equal to 8 or 1), consistent with Lemma 3; the interleaved sequence F''_n (base $B = 10$, digit blocks concatenating the six values) has period exactly $8 = \pi(3)$ and is injective on that period for this example.

Where the name comes from. Write the six component sequences of Example 4 stacked as rows, or interleaved together, one below the other, each read left to right through one period:

$$\begin{aligned} F_{n,1,1}: & \quad 01120221 \\ F_{n,2,2}: & \quad 02210112 \\ F_{n+1,3,3}: & \quad 00000000 \\ F_{n+1,4,4}: & \quad 01120221 \\ F_{n+1,5,5}: & \quad 02210112 \\ F_{n+2,6,6}: & \quad 00000000 \end{aligned}$$

F''_n is built by using/reading a *diagonal* slice through this stacked block, one digit contributed from each row, at a column position that advances (by the shift J) as we move down through the rows, rather than a single vertical slice straight down one column. This distinction is the point of the construction, and the reason for its name: a vertical read straight down any one column repeatedly lands on rows 3 and 6, which are identically 0 throughout, contributing no information that could distinguish one value of n from another; two of the six “lanes” are completely silent for a vertical read. A diagonal read, by contrast, zig-zags across all six rows as it advances, so the digits it picks up at each step are drawn from a different row each time, only occasionally landing on one of the silent all-zero rows and never landing on the same row twice in a single pass, pooling the non-constant information spread across the different component sequences rather than repeatedly sampling whichever row happens to be constant. This diagonal, zig-zagging read is the informal picture behind the formal interleaving $Z_{J,n}$ of (11): the shift J plays the role of the diagonal’s downward step, ensuring that the value contributed by each tooth is drawn from a different phase of a different component sequence rather than always the same one. We stress this is an intuition for the construction and the source of its name, not a proof of Conjecture 5: making this diagonal-sampling advantage rigorous, and showing exactly how large U must be for it to guarantee injectivity, is precisely the open question the conjecture asks.

Example 4 confirms the construction is arithmetically correct and gives one instance where the combined period equals $\pi(m)$ exactly; it does not by itself establish this in general, or say how U must scale with m . We isolate that as the paper’s key open condition.

Conjecture 5 (Zig-Zag Injectivity Conjecture). *There exists a polynomial $U(m) = O((\log m)^c)$, for some constant c , such that taking the summation bound in (12) at $U(m)$ makes $F_n'' \bmod (B^{3(U(m)+1)})$ injective as a function of $n \bmod \pi(m)$, for a suitable choice of base $B \geq m$ and constants $c_{K,n}$. (By the remark following Lemma 3, this is equivalent to the true period of F_n'' being exactly $\pi(m)$, not a proper divisor.)*

We have verified Conjecture 5 numerically in examples including the one above, with U growing slowly (logarithmically in the tested range) as m increases; we regard this as evidence for, not a proof of, the conjecture, and we state all downstream complexity results explicitly conditional on it.

Remark 6 (On the status of the logarithmic growth rate). *Here we discuss how the bound $U(m) = O((\log m)^c)$ in Conjecture 5 was arrived at: it was observed empirically, from the growth of the smallest U that worked in the examples we tried, and not derived from any argument that explains why it should be logarithmic. We only have empirical evidence that predicts this rate in advance or accounts for it after the fact. It is not even a matter of the codomain being large enough: since $B \geq m$, the range $B^{3(U(m)+1)}$ of F_n'' already exceeds $\pi(m) = O(m)$ at $U = 0$, so having enough distinct output values available is never the bottleneck: the open question is squarely whether the specific interleaving (12) actually realizes that available room injectively, and why a slowly growing number of teeth would suffice for that. We flag this gap explicitly rather confirmed with empirical evidence: understanding why a slowly growing U appears to suffice, if it does in general, is, in our view, one of the most important open question this paper raises, more so than the conjecture's truth alone.*

3.4 Reversibility

Lemma 7. *If, for fixed J , $Z_{J,n}$ as defined in (11) is computable by a classical algorithm running in time polynomial in $\log n$ and $\log m$, then it is computable by a reversible (unitary) quantum circuit of polynomial size, via the standard Bennett compute–copy–uncompute construction.*

This reduces the question of a polynomial-size unitary implementation of F_n'' to the question of a polynomial-time classical algorithm for each $F_{n+J,\cdot,\cdot}$, which is immediate: by the matrix identity

$$\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n \begin{pmatrix} F_1 \\ F_0 \end{pmatrix} \pmod{m}, \quad (13)$$

each (a, b) -Fibonacci term reduces to computing an n -th matrix power mod m by repeated squaring in $O(\log n)$ multiplications, each $\text{poly}(\log m)$ time, exactly the classical ingredient Bennett's construction needs. This resolves reversibility in principle; we now make the resulting circuit explicit, rather than leaving it as an existence statement.

3.4.1 An explicit reversible circuit for the matrix-power step

The classical repeated-squaring algorithm above can be made unitary by adapting the binary-exponentiation-in-superposition construction of [1]: there, an analogous identity

$$B^k = \prod_{j=0}^{\text{len}-1} (k_j(B^{2^j} - I_S) + I_S) \quad (14)$$

(eq. 42 in [1]) is used to compute the k -th power of an adjacency matrix B for k held in an ancilla register in superposition, using a Quantum Multiply operator U_{QM} built from a reversible modular read/write addressing scheme (`qarrayread/qarraywrite`) on a matrix interleaved into a single integer register, with circuit depth $O(\log k)$. The `qarrayread/qarraywrite` and U_{QM} were first introduced in [1] as a new way to do matrix arithmetic on a quantum computer in a new memory model given in [1]. Note that this method of binary exponentiation is newer and different to the decades old method of binary exponentiation for example found in Shor's algorithm. The same identity applies verbatim to our setting with $B = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \bmod m$ and the exponent register holding n instead of their path-length register holding k : precompute $B^{2^0}, B^{2^1}, \dots, B^{2^{\lceil \log_2 n \rceil - 1}} \bmod m$ classically (each an ordinary $O(\log n)$ -step repeated-squaring computation, done once, independent of the superposition), then apply U_{QM} controlled on each bit of the n -register to accumulate the product in superposition, exactly as in their eq. (30)–(31).

One simplification applies in our setting that does not apply to their general- V construction: here B is always a fixed 2×2 matrix ($V = 2$ is a constant, not a parameter growing with the problem size), so the general-purpose single-large-integer `qarrayread/qarraywrite` addressing scheme of [1], designed to avoid QRAM overhead for large $V \times V$ matrices, is what we borrow here with the following specialisation; used for 2 by 2 matrices instead of V by V , a fixed constant number (4) of $O(\log m)$ -qubit registers for the matrix entries suffices, addressed directly rather than via modular bit-extraction. What we also do borrow, for the quantum matrix implementation for this paper, is the *binary-exponentiation-identity circuit* of [1] too: applying U_{QM} (specialized to 2×2 modular matrix multiplication) controlled on each of the $O(\log n)$ bits of n , giving overall circuit depth $O(\log n)$, each layer being one modular 2×2 matrix multiplication (four modular multiplications of $O(\log m)$ -bit integers, each $O((\log m)^2)$ time by usual multiplication, or $O(\log m \log \log m)$ with fast multiplication), for a total gate count of $O(\log n \cdot (\log m)^2)$ (usual) or better, comfortably polynomial in $\log n$ and $\log m$ as required by Theorem 11.

Remark 8 (Illustrative value of the constant- V instance). *The general framework of [1] is designed around a matrix size V that grows with the problem instance (there, the number of graph vertices), which is what makes the single-large-integer arithmetic-encoding technique (`qarrayread/qarraywrite`) worth its complexity: it avoids QRAM-style addressing overhead precisely when V is large and variable. The present setting "is the opposite", and arguably a useful complementary case to have worked out: $V = 2$ is fixed once and for all, so the encoding machinery collapses one of the simpler possible instance (Remark above), while the other half of the framework ; the binary-exponentiation-identity circuit that computes B^k (or here, $A^n \bmod m$) via a polynomial number of controlled applications of U_{QM} ; remains*

exactly as essential as in the general- V case, since it is what keeps the exponent computation polynomial in $\log n$ regardless of V . In that sense this instantiation isolates and still uses the exponentiation half of [1]’s construction on its own, independent of the array-addressing half, which may make it a useful minimal worked instance/example of that framework in its own right, beyond its role here in resolving the reversibility gap in the proof of Theorem 11.

This reduces the question of a polynomial-size unitary implementation of F''_n to the question of a polynomial-time classical algorithm for each $F_{n+J,\cdot,\cdot}$, which is immediate by the above. It does not, on its own, bound $U(m)$, which is a separate combinatorial question addressed by Conjecture 5 alone.

Remark 9 (The golden-mean construction of Section 2 is *not* used here). *The paper gives two independent routes to Fibonacci-type quantities, and they are not interchangeable. Reproducing $F_{n,a,b}$ via the Binet-type formula (3) requires φ to absolute precision $\varepsilon = O(1/(n\varphi^{n-1}))$, i.e. $\Theta(n)$ bits. In the period-finding algorithm of this section, n must range over at least one full Pisano period, so $n = \Theta(m)$, forcing $\Theta(m) = \Theta(2^{\log m})$ bits of φ ; exponential in the input size $\log m$, on top of the fact that the Section 2 construction is itself exponential-time (Remark 2). Using it here would immediately break Theorem 11, independently of Conjecture 5. This is exactly why each $F_{n+J,\cdot,\cdot}$ term in (11) is instead computed by the integer matrix-power identity above, which never introduces φ or any real-valued approximation: every quantity computed in the proof of Theorem 11 is an exact integer mod m . The Grover-based construction of Section 2 is included solely as an independent, self-contained result about evaluating φ , Catalan’s constant, and the beta function. The Catalan’s constant algorithm here plays no role in, and is not required by, the period-finding algorithm of Section 4.*

Remark 10 (Two variants of the period-finding algorithm). *Combining Sections 2-4 as stated yields two distinct instantiations of the period-finding algorithm, depending on which subroutine is used to generate (a,b) -Fibonacci values inside (11):*

- **Variant A (Grover-based, exponential time).** *Each Fibonacci value is obtained from φ via Binet’s formula (3), with φ computed by the Section 2 construction. By Remark 2 together with the precision argument above, this variant runs in exponential time, even though the quantum Fourier transform step itself is polynomial; the exponential cost is entirely absorbed in the golden-mean subroutine.*
- **Variant B (matrix-based, polynomial time).** *Each Fibonacci value is obtained by exact modular matrix exponentiation, as described above. This is the variant used in the proof of Theorem 11, and the only variant for which we claim a polynomial-time result.*

We record Variant A because it is the more direct route from Section 2’s construction, and because we regard it as conceptually interesting in its own right despite giving no speedup - a quantum algorithm need not outperform the best classical algorithm to be of interest; e.g. see [7], which the authors likewise present as conceptually novel algorithm rather than as a runtime improvement over any comparable classical algorithm. All complexity claims in this paper (Theorem 11 and its corollary) refer to Variant B only.

4 Quantum Algorithm for Pisano Periods

Theorem 11 (Quantum Zig-Zag Theorem, conditional). *Assume Conjecture 5. Then there is a quantum algorithm that, given m , computes the Pisano period $\pi(m)$ in time and space polynomial in $\log m$.*

Sketch. **Step 1 (classical).** Fix $B \geq m$ and the constants $c_{K,n}$; choose $U = U(m)$ as in Conjecture 5. Since $\pi(m) \leq 6m$ for all m (a standard, unconditional fact about Pisano periods), we have $p := \pi(m) = O(m)$.

Step 2 (quantum). Choose the first-register size t with

$$t = \lceil 2 \log_2 m \rceil + c \quad (15)$$

for a small constant c (e.g. $c = 3$, as in the standard Shor-algorithm register-size argument), so that $N := 2^t \geq 2p^2$. This ensures the register holds $M = N/p = \Theta(m)$ full copies of the period, which is what the continued-fraction step (Step 6) needs to recover p exactly from a single measurement with constant probability; note that M copies of the period are represented in *one* superposition over $t = O(\log m)$ qubits, not via M sequential repetitions of anything, so this choice does not cost extra time or qubits beyond $t = O(\log m)$. Prepare $\frac{1}{\sqrt{N}} \sum_{n=0}^{N-1} |n\rangle|0\rangle$ and apply the reversible circuit for F_n'' (Section 3.3) to obtain $\frac{1}{\sqrt{N}} \sum_n |n\rangle|F_n''\rangle$.

Step 3. Measure the second register, obtaining some value k ; by Lemma 3 and Conjecture 5 together, F_n'' has period exactly $p := \pi(m)$ (not a divisor or multiple of it), so the surviving superposition in the first register is $\frac{1}{\sqrt{M}} \sum_{w=0}^{M-1} |u + wp\rangle$, where u is the smallest n with $F_n'' = k$ and $M = N/p = \Theta(m)$ as set in Step 2.

Step 4. Apply the QFT to the first register (size N , so this costs $O(t^2) = O((\log m)^2)$ gates).

Step 5. Measure; the standard Shor-type analysis, using $N \geq 2p^2$ from Step 2, shows the observed value v satisfies $v/N \approx L/p$ for some integer L , with the continued-fraction convergent of v/N recovering $p = \pi(m)$ exactly with probability $\Omega(1/\log \log p)$ on a single run.

Step 6 (classical). Recover $p = \pi(m)$ from v/N via the continued-fraction algorithm, in time polynomial in t .

Step 7 (classical). Because a single run succeeds with only constant-ish (not certain) probability, repeat Steps 2–6 a polynomial number of times (standard probability amplification, as in Shor’s algorithm) and keep the candidate satisfying the known necessary properties of a Pisano period (e.g. parity constraints); by Lemma 3 no factoring step is needed here, since –, unlike in Shor’s original factoring algorithm, where the measured quantity can genuinely be a proper divisor of the order sought, the period recovered here is exactly $\pi(m)$ whenever Conjecture 5 holds for the chosen U , not merely a divisor requiring further processing. This is the polynomial-number-of-trials step; it is distinct from, and in addition to, the register-size choice of Step 2, which concerns copies of the period within a single run rather than repeated runs.

Every step above other than the injectivity guarantee in Step 3 is a standard, unconditional fact about quantum period finding (Steps 2, 4–6) or an classical computation (Steps 1, 3, 7); the reversible circuit of Step 2 is polynomial-size by Section 3.3, and the register size

$t = O(\log m)$ chosen in Step 2 keeps every step polynomial in $\log m$ despite N itself being as large as $\Theta(m^2)$. Given Conjecture 5, all steps run in time and space polynomial in $\log m$, proving the theorem. $\square$

Remark 12. *This construction generalizes to (a, b) -Fibonacci sequences in general (Pell numbers being the case $(a, b) = (2, 1)$) by replacing $F_{n+J, \cdot}$ in (11) with the corresponding (a, b) -shifted terms, and to the rank and order of m (the number and position of zeros in a period) by a Grover search over the second register of the state in Step 2 for entries beginning with 0.*

Remark 13 (An unconditional two-term instance). *The Zig-Zag construction of Section 3 interleaves many shifted (a, b) -Fibonacci terms together, using more teeth than strictly necessary in order to make injectivity plausible without pinning down exactly which terms suffice; that is what makes Conjecture 5 an open question rather than a proof. There is a second, much more minimal way to get an injective interleaving, which turns out to be provable outright, with no conjecture needed. Rather than interleaving several different (a, b) -sequences, interleave the standard Fibonacci sequence together with its own immediate successor: define*

$$G_n := (F_n \bmod m) + m \cdot (F_{n+1} \bmod m) \in [0, m^2), \quad (16)$$

i.e. the base- m two-digit encoding of the pair $(F_n \bmod m, F_{n+1} \bmod m)$.

Lemma 14. G_n is periodic in n with period exactly $\pi(m)$, and is injective on $n = 0, 1, \dots, \pi(m) - 1$.

Proof. Write $v_n = \begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} \bmod m = A^n v_0 \bmod m$ with $v_0 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$, as in Lemma 3; there it was shown that v_n has period exactly $\pi(m)$. Since G_n is, by construction, simply the base- m digit pairing of the two coordinates of $v_n \bmod m$ (each already reduced to $[0, m)$ before being interleaved), G_n has the same period as v_n , namely $\pi(m)$ exactly, and $G_i = G_j$ if and only if $v_i = v_j$ (as the base- m pairing of two digits in $[0, m)$ is a bijection onto $[0, m^2)$). Suppose $G_i = G_j$ for some $0 \leq i < j < \pi(m)$; then $v_i = v_j$, i.e. $A^i v_0 \equiv A^j v_0 \pmod{m}$. Since $\det A = -1$ is a unit mod m , A is invertible mod m , so multiplying by $(A^{-1})^i$ gives $v_0 \equiv A^{j-i} v_0 \pmod{m}$ with $0 < j - i < \pi(m)$, contradicting the minimality of $\pi(m)$. Hence G_n is injective on $\{0, \dots, \pi(m) - 1\}$. $\square$

Since G_n is computed directly from $(F_n \bmod m, F_{n+1} \bmod m)$, which is exactly the pair already produced by the matrix-power circuit of Section 3.4.1 (that circuit, being built from the novel Arithmetic-Encoding / binary-exponentiation framework given in [1], natively works with non-negative integers, and G_n itself is a non-negative integer built from the same two already-computed non-negative residues, so no new circuit machinery beyond Section 3.4.1 is required), substituting G_n for F_n'' throughout the proof of Theorem 11 yields the same conclusion ; a quantum algorithm computing $\pi(m)$ in time and space polynomial in $\log m$; unconditionally, with no dependence on Conjecture 5.

We nonetheless present the Zig-Zag construction of Section 3 (which in our opinion is more interesting) as the paper's main construction, rather than leading with this two term pairing: the multi-term, multi-tooth interleaving of several distinct (a, b) -Fibonacci sequences is closer in spirit to, and a more informative illustration of, the general digit-block interleaving idea this paper introduces, of which the two-term pairing above is only the smallest possible

instance ($U = 0$, unshifted, unscaled). We record it here as an unconditional companion result and as a unconditional proof that a lower term example of the general construction, not as a replacement for it.

4.1 The Advantage of the general Zig-Zag construction

Here are four reasons for the advantages of the the multi-tooth Zig-Zag construction compared with the "two term", unconditional G_n .

1. **Generality as a template.** The clean unconditional proof for G_n leans on a fact special to first-order linear recurrences: $\det A = -1$ is a unit modulo every m , which is what makes A invertible mod m for all m and drives the whole argument. Higher-order linear recurrences (Tribonacci-type sequences, or general order- r recurrences) need not have an analogous companion matrix that is unimodular, or an analogous two-term pairing that works uniformly in m . The many-tooth Zig-Zag interleaving does not lean on this special structure, and is intended as the more portable technique should one want to extend this approach beyond the ordinary Fibonacci recurrence.
2. **Slack.** G_n 's injectivity argument is tight: it uses exactly the two terms F_n, F_{n+1} and no more, with no room to spare if some unforeseen obstruction were found. Zig-Zag's conjectured $U(m)$ can be taken larger than any conjectured minimum, giving room for injectivity to hold even if the true required number of teeth turns out to be somewhat larger than currently conjectured.
3. **A possible route to noise resilience (sketch, non-rigorous).** Rather than interleaving $U + 1$ different (a, b) -sequences, one could instead use the $U + 1$ teeth as *redundant* copies of the same computation, so $U + 1$ independently compiled circuits computing the same $(F_n \bmod m, F_{n+1} \bmod m)$, ideally on disjoint qubit subsets so their errors are uncorrelated; and add a classical majority-vote filter after measurement, accepting a run only if more than half the teeth agree. Heuristically, if each tooth fails independently with probability q , the majority-vote fails only when more than half the teeth fail simultaneously, with probability roughly $\binom{U+1}{\lceil U/2 \rceil} q^{\lceil U/2 \rceil}$; falling exponentially in U at the cost of only a polynomial ($U + 1$ -fold) increase in circuit size, in the spirit of a classical repetition code. We stress this is a sketch, not a claim: it requires the extra circuitry's own added failure probability to be outweighed by this gain, which is a genuine trade-off that would need to be worked out rigorously under a specific noise model, and is not something the construction as currently presented in this paper does. We record it here only as a plausible reason the general multi-tooth structure could matter beyond injectivity, not as an established property of Theorem 11.
4. As an new interesting illustrative example of the low QRAM memeory model for matrix multiplication and polynomial time and polynomial space matrix exponentiation of matrices, first introduced in the paper [1], for solving the problem of proving that a degree 5 or greater degree polynomial has no genral formula in radicals, that is a new proof of the Abel Ruffini Theroem, and that work adapted here as a solution for the

Fibonacci matrix implementation part of this paper. Hence resulting in an interesting new application of that QRAM memory model for matrix multiplication/matrix exponentiation.

Corollary 15. *Assume Conjecture 5. Given a candidate prime p , whether p is a Wall–Sun–Sun prime ($p^2 \mid \pi(p)$) or a near-WSS prime (satisfying (1) for some A) can be decided in time polynomial in $\log p$.*

Proof. Compute $\pi(p)$ by Theorem 11 and test the corresponding divisibility or congruence classically in $\text{poly}(\log p)$ time. $\square$

We emphasize that this is a fast test for a single, given candidate p for being a Wall–Sun–Sun prime. A search speedup for Wall–Sun–Sun primes would require, e.g., stacking Grover amplification over candidates with this test as the sub-oracle, which we do not develop here. Since no Wall–Sun–Sun primes are currently known, we present this as a faster per-candidate test rather than a claim that discovery is imminent. If we use the remark 13 (an unconditional two term instance/example) a quantum algorithm that can test for a Wall–Sun–Sun prime in quantum polynomial time and quantum polynomial space.

5 Conclusion

We have given a quantum algorithm for the Pisano period $\pi(m)$, built by transforming the (non-injective) Fibonacci recurrence mod m into an auxiliary Zig-Zag function interleaving several shifted (a, b) -Fibonacci sequences by place value. The resulting polynomial-time claim (Theorem 11) is conditional on the Zig-Zag Injectivity Conjecture (Conjecture 5), for which we gave numerical evidence in one worked family of examples but no general proof; testing candidate Wall–Sun–Sun and near-Wall–Sun–Sun primes follows as a direct corollary. Independently, we described a new hybrid classical–quantum construction, based on a non-standard use of Grover’s algorithm combined with two-dimensional Weyl equidistribution, for evaluating integral–series representations of φ , Catalan’s constant, and the beta function; we were explicit that this construction is not a speedup over classical quadrature, and offered it instead as a new way of extracting real-valued quantities from Grover’s algorithm. We consider proving Conjecture 5, or replacing the Zig-Zag construction with one whose injectivity can be established unconditionally (e.g. via an explicit bound from the structure of (a, b) -Fibonacci recurrences modulo m), the main open problem raised by this work. We note, as a companion result recorded in Remark 13, that a two-term instance of the same interleaving idea ($G_n = (F_n \bmod m) + m \cdot (F_{n+1} \bmod m)$) already gives an unconditional version of Theorem 11, requiring no conjecture; we present the general Zig-Zag construction as the paper’s main contribution because it better illustrates the underlying digit-interleaving idea and its associated quantum circuit machinery, of which the two-term case is only the minimal instance.

A Trigonometric Identities Used in Section 2

We record the identities used to build the cosine/sine lookup tables of Section 2, for a single Grover instance with rotation angle θ and iteration count k ; the second instance is identical with θ, k replaced by θ_2, k_2 .

From $2 \cos^2 \gamma - 1 = \cos 2\gamma$, applied with $\gamma = \frac{(2k+1)\theta}{2}$,

$$\cos((2k+1)\theta) = 2 \cos^2\left(\frac{(2k+1)\theta}{2}\right) - 1, \quad (17)$$

where the right-hand side is exactly the quantity Quantum Phase Estimation returns: after k Grover iterations the state has overlap $\cos\left(\frac{(2k+1)\theta}{2}\right)$ with the non-solution subspace, and QPE applied to the Grover operator reads off the corresponding phase directly from the quantum state, without any classical sampling or frequency estimation.

For the sine values, rotate the solution state by an additional $\mu = \lfloor \frac{\pi}{4} \sqrt{N} \rfloor / 2$ Grover iterations (approximately $\pi/2$) and use $\cos(\beta + \pi/2) = \sin \beta$:

$$\sin((2k+1)\theta) = 2 \cos^2\left(\frac{(2k+1+2\mu)\theta}{2}\right) - 1. \quad (18)$$

Combining both, for a pair (k_1, k_2) with one index from each instance, via the angle-addition formula gives (9). The node point itself is approximated by the classical Diophantine search of Step 2–3 of Section 2.3: the classical machine scans all κ^2 pairs (k_1, k_2) against the two length- κ tables built in Step 1, using the two-dimensional Weyl equidistribution argument of Remark 2 to guarantee that, for κ large enough, some pair gives $(\Phi_{1,k_1} + \Phi_{2,k_2}) \bmod 2\pi$ within any desired δ of the target node x_i . The same identities, specialized to a single free parameter and equally spaced nodes, give the beta-function and Catalan’s-constant evaluations; we omit the repetition.

References

- [1] *A Constructive Quantum Graph Theoretic Spectral Proof of the Abel–Ruffini Theorem*, 2026 (companion paper; see §1.2, “Quantum Bi-Partite Graph Hamiltonian Cycle Finding Using Matrix Traces (QHCFUMT),” for the Arithmetic-Encoding matrix representation, the `qarrayread/qarraywrite` operators, and the binary-exponentiation-in-superposition identity used in Section 3.4.1 of the present paper).
- [2] Wikipedia, *Wall–Sun–Sun Prime*, retrieved 5 May 2025.
- [3] B. Benfield and O. Lippard, *Connecting Zeros in Pisano Periods to Prime Factors of k -Fibonacci Series*, arXiv:2407.20048, 30 Jan 2025.
- [4] P. J. Nahin, *Inside Interesting Integrals*, Springer, 1st edition, 2015.
- [5] Wikipedia, *Beta Function*, retrieved 5 May 2025.
- [6] Wikipedia, *Generalizations of Fibonacci Numbers*, retrieved 2 June 2025.
- [7] 2013, Locate ref